Retrieval-Augmented Generation for Healthcare Documents

A Proposed NLP Pipeline for Context-Aware Document Question Answering

Hatim Maula

Registration No. 23BCE2238



ABSTRACT

Flow: ***Healthcare Documents → Manual / Keyword Search → RAG → Context-Grounded Answer***

SUMMARY

Healthcare documents are difficult to query efficiently using manual or keyword-based search. Retrieval-Augmented Generation (RAG) retrieves relevant passages and provides them as context to an LLM, with the goal of context-grounded natural-language question answering over healthcare documents.

PROBLEM STATEMENT



Healthcare Documents

Long and information-dense



Manual Search

Slow and difficult at scale



Keyword Search

Limited semantic understanding



Direct LLM

May not be grounded in the document

THE PROBLEM

How can an NLP system retrieve semantically relevant information from healthcare documents and use it to generate context-grounded answers to natural-language queries?

OBJECTIVES



Process healthcare documents



Retrieve relevant passages



Provide retrieved context to an LLM



Generate grounded answers



Keyword retrieval — exact lexical overlap



Embedding retrieval — captures semantic similarity between query and passage

INTRODUCTION

TRADITIONAL LLM

- **User Query**
- **LLM (parametric memory only)**
- **Answer**

Risk of hallucination — no grounding

RAG

- **User Query**
- **Retrieve Relevant Passages**
- **LLM + Retrieved Context**
- **Grounded Answer**

Response grounded in retrieved content

Key idea: ***Retrieve first → Generate second***

RAG COMPONENTS

Document → Embedding → Vector Space ← Query Embedding

Retriever
→ Finds relevant document chunks

Embedding Model
→ Converts text/query into dense vectors

Vector Database
→ Stores embeddings for similarity search

LLM
→ Generates the final answer from context

LITERATURE SURVEY

Study	Approach & Key Contribution	Relation to Our Project
Karpukhin et al., 2020 Dense Passage Retrieval (DPR) — EMNLP	Dual-encoder architecture: separate query/passage encoders map text to dense embeddings; retrieval by semantic similarity rather than lexical overlap. Outperformed BM25 on open-domain QA.	Establishes embedding-based retrieval as the foundation for our vector-search step.
Lewis et al., 2020 Retrieval-Augmented Generation — NeurIPS	Retriever + generator architecture: a neural retriever selects passages that condition a seq2seq generator's output, coining the "RAG" architecture.	Directly inspires our retrieve-then-generate pipeline design.
Izacard & Grave, 2021 Fusion-in-Decoder (FiD) — EACL	Fusion-in-Decoder: each retrieved passage is encoded separately and evidence is fused inside the decoder, improving accuracy as more passages are combined.	Informs how our LLM will combine several retrieved chunks into one answer.
Zakka et al., 2023/24 Almanac — NEJM AI	Retrieval-augmented LLM grounded in curated clinical guideline documents via semantic retrieval; improved factuality and safety versus standalone LLMs.	Confirms the value of RAG specifically for grounding answers in healthcare documents.
Xiong et al., 2024 MIRAGE Benchmark — ACL Findings	Systematic benchmark evaluating how corpus, retriever, and LLM choices affect RAG accuracy on medical QA.	Guides our design choices for chunking, retrieval, and corpus in the healthcare domain.
Our Proposed System (this project)	Applies established RAG concepts to a healthcare-document Q&A pipeline: chunking, dense embeddings, vector retrieval, LLM generation.	Extends the reviewed work into a focused, class-project-scale healthcare RAG pipeline.

Progression: Dense Retrieval → RAG → Multi-Passage RAG → Healthcare RAG → Medical RAG Evaluation

PROPOSED METHODOLOGY

Pipeline: ***Raw Text → Preprocessing → Chunking → Embeddings → Semantic Retrieval → Generation***



Text Processing

Clean and prepare extracted healthcare text.

Input: PDF / text

→ extraction → cleaning → normalization

Chunking

Divide large documents into smaller searchable passages.

Why chunk: focused, retrievable evidence

Document → C_1 | C_2 | C_3 | …

(chunk size + overlap, tuned later)

Embeddings

Convert text into numerical representations of semantic meaning.

Dense vector rep.

"patient has fever" →

[0.12, -0.43, 0.87, …] (conceptual)

Semantic Retrieval

Compare the query with document representations to find relevant passages.

Query vector ↔ Document vectors

Similarity → Top-k relevant chunks

Language Generation

Use the retrieved context to generate a natural-language response.

Query + Retrieved Context → Prompt

→ LLM → Answer

PROPOSED SYSTEM ARCHITECTURE

PROPOSED / INITIAL STACK

OFFLINE / DOCUMENT INDEXING

- Healthcare PDFs / Text
- Text Extraction (PyMuPDF)
- Preprocessing & Chunking (Python)
- Embeddings (MiniLM-L6-v2)
- Vector Database (FAISS)

feeds retrieval

METADATA / SOURCE MAPPING

SQLite — document_id · chunk_id · page · source

Purpose: convert documents into searchable vectors, with SQLite mapping each chunk back to its source document and page.

ONLINE / QUERY & GENERATION

via Streamlit UI — upload · query · results

- User Query
- Query Embedding (Sentence Transformer)
- Similarity Search (FAISS)
- Top-k Relevant Chunks
- Context + Query
- LLM (Ollama, local)
- Grounded Answer

Purpose: retrieve the most relevant evidence for each query and use it as generation context.

FAISS = vector retrieval **SQLite** = metadata / source mapping **LLM** = answer generation **Streamlit** = user interface

DOCUMENT INGESTION + QUERY / RETRIEVAL WORKFLOW

A. DOCUMENT INGESTION — OFFLINE

PDF → PyMuPDF → Extracted Text → Cleaning → Chunks → Embeddings → FAISS

SQLite side mapping (per chunk): document_id · chunk_id · page · source — the document is processed once into a searchable representation.

B. QUERY / RETRIEVAL — ONLINE

User Query → Query Embedding → FAISS Search → Top-k Chunks → Metadata Lookup → Context + Prompt → LLM → Answer

Prompt = System Instruction + Retrieved Context + User Query → output returned as Answer + Source / Page

Vector database retrieves evidence. LLM generates the answer.

EXPECTED EVALUATION & FUTURE SCOPE

Planned Evaluation

No results yet — Review 1 presents planned metrics only.

RETRIEVAL

- Precision@k
- Recall@k
- Retrieval relevance

GENERATION

- Answer relevance
- Faithfulness / groundedness
- Completeness

SYSTEM

- Retrieval latency
- End-to-end response time

Metrics will be measured after implementation and testing.

Future Scope

- Improve chunking strategy
- Compare embedding models
- Hybrid keyword + semantic retrieval
- Reranking of retrieved chunks
- Stronger source / citation display
- Larger healthcare document collections

FROM LONG DOCUMENTS TO GROUNDED ANSWERS

CONCLUSION

- **Problem:** Healthcare documents are difficult to search manually.
- **Approach:** RAG combines semantic retrieval with LLM generation.
- **Proposal:** Apply the pipeline to healthcare-document question answering.

Healthcare Documents → NLP Processing → Semantic Retrieval → Relevant Context → LLM → Grounded Answer

Thank You — Questions?